INSIDE STLC ACADEMY

PRACTICAL TEST AUTOMATION WITH PLAYWRIGHT

API Testing with Playwright Template

Test APIs directly and combine them with UI tests.

Introduction

Playwright can test APIs directly via request — faster and more stable than the UI, and great for setting up state. This template covers standalone API tests and using the API to speed up UI tests.

Instructions

- Use the request fixture for API calls.
- Assert status, body and key fields.
- Use API calls to set up data before UI tests.

Worked Example (standalone API test)

- test('creates a user', async ({ request }) => {
- const res = await request.post('/api/users', { data: { name: 'Test' } });
- expect(res.status()).toBe(201);
- const body = await res.json();
- expect(body.name).toBe('Test');
- });

Using API to Seed UI Tests

Create the data you need via request.post (fast, reliable), then drive only the behaviour under test through the UI. This avoids slow, brittle setup clicks.

Blank Reusable Template

- Endpoint:
- Method:
- Request data:
- Expected status:
- Expected body fields:

- Cleanup needed?

Practical Exercise

Write one API test against a public API (assert status + a field), then write a UI test that uses an API call to set up its starting state.

Best Practices

- Assert status and the fields that matter.
- Use API setup to keep UI tests focused and fast.
- Clean up data you create where possible.

Common Mistakes

- Doing all setup through slow UI clicks.
- Only checking status, never the body.
- Leaving test data behind and polluting environments.

> **INSIDE STLC PRO TIP**
>
> The fastest UI tests do their setup via the API. Reserve the UI for the behaviour you are actually testing — everything else is faster and more stable through request.